Day 1 – Problem Solving Techniques and Data Structures – I

Set 2 – Insurance Claims Processing

Business Scenario	An insurer must route claims through approval stages, detect duplicate claim submissions and monitor claim spikes after a regional event.
Tasks	**1.** Create a BST of claim IDs with insert and search; show the three traversals and explain which traversal gives claims in sorted order. **2.** Use hashing to detect duplicate or near-duplicate claims (same policy number and incident date, differing only in letter case or spacing); explain how collisions are handled. **3.** Model claim approval stages (registration, assessment, surveyor, settlement) as a directed graph and find the shortest path between two stages using BFS; detect loops using DFS. **4.** Find the longest run of consecutive days on which claims exceeded 100 using a sliding window. **5.** Using two pointers, merge two sorted claim lists (from two regional offices) into a single sorted list without extra sorting.
Deliverables	Working code, test cases covering empty input and duplicates, and a table of complexity per solution.

Task 1: Binary Search Tree (BST) of Claim IDs

Sample Input:

Claim IDs (insertion order): 5012, 3008, 7104, 2001, 4020, 6055, 8099, 3550

Program:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root

def search(root, key):
    while root is not None:
        if key == root.key:
            return True
        elif key < root.key:
            root = root.left
        else:
```

```
            root = root.right
    return False

def inorder(root):
    if root is None:
        return []
    return inorder(root.left) + [root.key] + inorder(root.right)

def preorder(root):
    if root is None:
        return []
    return [root.key] + preorder(root.left) + preorder(root.right)

def postorder(root):
    if root is None:
        return []
    return postorder(root.left) + postorder(root.right) + [root.key]

claim_ids = [5012, 3008, 7104, 2001, 4020, 6055, 8099, 3550]
root = None
for cid in claim_ids:
    root = insert(root, cid)

print("Search 4020:", search(root, 4020))
print("Search 9999:", search(root, 9999))
print("Inorder:", inorder(root))
print("Preorder:", preorder(root))
print("Postorder:", postorder(root))
```

Output:

```
Search 4020: True
Search 9999: False
Inorder: [2001, 3008, 3550, 4020, 5012, 6055, 7104, 8099]
Preorder: [5012, 3008, 2001, 4020, 3550, 7104, 6055, 8099]
Postorder: [2001, 3550, 4020, 3008, 6055, 8099, 7104, 5012]
```

Explanation: In a BST every smaller ID is stored in the left subtree and every larger ID in the right subtree. insert() walks down from the root and places the new ID at the first empty position (a duplicate ID is ignored). search() follows the same left/right decisions and stops when the ID is found or the tree ends. Inorder traversal visits left subtree, node, right subtree, so it prints the claim IDs in ascending (sorted) order; this is the traversal that gives claims in sorted order. Preorder (node first) is useful for copying a tree and Postorder (node last) for deleting one.

Complexity: insert and search take O(log n) on average and O(n) in the worst case (a skewed tree); each traversal takes O(n) time; space is O(n) for the nodes plus O(h) for the recursion stack.

Task 2: Duplicate Claim Detection Using Hashing

Sample Input:

Claims (ClaimId, PolicyNo, IncidentDate):

ClaimId	PolicyNo	IncidentDate
1001	POL-4521	2026-10-05
1002	pol-4521	2026-10-05
1003	POL -4521	2026-10-05
1004	POL-7788	2026-10-06
1005	POL-4521	2026-10-07
1006	pol-7788	2026-10-06
1007	POL-9020	2026-10-06

Program:

```
def normalize(policy, date):
    policy = "".join(policy.split()).upper()
    date = "".join(date.split())
    return policy + "|" + date

class HashTable:
    def __init__(self, size=5):
        self.size = size
        self.buckets = [[] for _ in range(size)]

    def hash_function(self, key):
        total = 0
        for ch in key:
            total = total * 31 + ord(ch)
        return total % self.size

    def add(self, key, claim_id):
        index = self.hash_function(key)
        for stored_key, first_id in self.buckets[index]:
            if stored_key == key:
                return first_id
        self.buckets[index].append((key, claim_id))
        return None

def find_duplicates(claims):
    table = HashTable()
    duplicates = []
    for claim_id, policy, date in claims:
        key = normalize(policy, date)
        original = table.add(key, claim_id)
        if original is not None:
            duplicates.append((claim_id, original))
    return duplicates, table

claims = [
    (1001, "POL-4521", "2026-10-05"),
```

```
    (1002, "pol-4521", "2026-10-05"),
    (1003, "POL -4521", "2026-10-05"),
    (1004, "POL-7788", "2026-10-06"),
    (1005, "POL-4521", "2026-10-07"),
    (1006, "pol-7788", "2026-10-06"),
    (1007, "POL-9020", "2026-10-06"),
]

duplicates, table = find_duplicates(claims)
for dup, original in duplicates:
    print("Claim", dup, "duplicates claim", original)

print("Buckets:")
for i in range(table.size):
    keys = [k for k, c in table.buckets[i]]
    print(" ", i, keys)
```

Output:

```
Claim 1002 duplicates claim 1001
Claim 1003 duplicates claim 1001
Claim 1006 duplicates claim 1004
Buckets:
  0 ['POL-4521|2026-10-07']
  1 []
  2 ['POL-7788|2026-10-06']
  3 ['POL-4521|2026-10-05', 'POL-9020|2026-10-06']
  4 []
```

Explanation: Each claim is first normalised: all spaces are removed and the policy number is converted to upper case, so "pol-4521" and "POL -4521" both become "POL-4521". The policy number and incident date are joined into one key and passed through a hash function that gives a bucket number. If the same key is already stored in that bucket, the claim is reported as a duplicate; otherwise it is stored. Claim 1005 has the same policy but a different date, so it is correctly not reported. Collisions (different keys landing in the same bucket) are handled by separate chaining: each bucket is a list, and the full key is compared with every entry in that list before deciding. In the output, bucket 3 holds two different keys (POL-4521|2026-10-05 and POL-9020|2026-10-06), yet they are not confused with each other.

Complexity: average O(1) per claim, so O(n) for n claims; the worst case is O(n) per claim ($O(n^2)$ overall) if every key lands in one bucket; space is O(n).

Task 3: Claim Approval Stages as a Directed Graph (BFS and DFS)

Sample Input:

Directed edges (stage → next stages):

Stage	**Can move to**
Registration	Assessment
Assessment	Surveyor, Settlement
Surveyor	Assessment, Settlement
Settlement	(none)

Program:

```
from collections import deque

graph = {
    "Registration": ["Assessment"],
    "Assessment": ["Surveyor", "Settlement"],
    "Surveyor": ["Assessment", "Settlement"],
    "Settlement": [],
}

def shortest_path(graph, start, goal):
    if start not in graph or goal not in graph:
        return None
    parent = {start: None}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]
        for nxt in graph[node]:
            if nxt not in parent:
                parent[nxt] = node
                queue.append(nxt)
    return None

def find_loop(graph):
    state = {}
    path = []

    def dfs(node):
        state[node] = "visiting"
        path.append(node)
        for nxt in graph[node]:
            if state.get(nxt) == "visiting":
                return path[path.index(nxt):] + [nxt]
            if nxt not in state:
                loop = dfs(nxt)
                if loop:
                    return loop
        path.pop()
        state[node] = "done"
        return None

    for node in graph:
        if node not in state:
            loop = dfs(node)
            if loop:
```

```
                return loop
    return None

print("Registration -> Settlement:", shortest_path(graph, "Registration", "Settlement"))
print("Registration -> Surveyor:", shortest_path(graph, "Registration", "Surveyor"))
print("Settlement -> Registration:", shortest_path(graph, "Settlement", "Registration"))
print("Loop found:", find_loop(graph))

acyclic = {
    "Registration": ["Assessment"],
    "Assessment": ["Surveyor"],
    "Surveyor": ["Settlement"],
    "Settlement": [],
}
print("Loop in acyclic graph:", find_loop(acyclic))
```

Output:

```
Registration -> Settlement: ['Registration', 'Assessment', 'Settlement']
Registration -> Surveyor: ['Registration', 'Assessment', 'Surveyor']
Settlement -> Registration: None
Loop found: ['Assessment', 'Surveyor', 'Assessment']
Loop in acyclic graph: None
```

Explanation: The stages are stored as an adjacency list in a dictionary, and each arrow is one-way (directed). BFS uses a queue and visits stages level by level, so the first time the goal stage is reached the path has the fewest steps; a parent dictionary is used to rebuild the path. DFS goes as deep as possible along one path and keeps a "visiting" mark on the stages of the current path; if an edge leads back to a stage that is still "visiting", a loop exists. Here Surveyor can send a claim back to Assessment, giving the loop Assessment → Surveyor → Assessment. No path exists from Settlement back to Registration because the edges are directed, so None is returned.

Complexity: BFS and DFS both take O(V + E) time and O(V) space, where V is the number of stages and E the number of arrows.

Task 4: Longest Run of High-Claim Days Using Sliding Window

Sample Input:

Daily claim counts (days 1 to 12): 80, 120, 130, 95, 101, 150, 160, 170, 100, 105, 110, 99

Program:

```
def longest_run(counts, limit=100):
    left = 0
    best_len = 0
    best_start = -1
    for right in range(len(counts)):
        if counts[right] <= limit:
            left = right + 1
        elif right - left + 1 > best_len:
            best_len = right - left + 1
            best_start = left
    if best_len == 0:
        return 0, -1, -1
```

```
    return best_len, best_start + 1, best_start + best_len

daily_claims = [80, 120, 130, 95, 101, 150, 160, 170, 100, 105, 110, 99]
length, first_day, last_day = longest_run(daily_claims)
print("Daily claims:", daily_claims)
print("Longest run:", length, "days")
print("From day", first_day, "to day", last_day)
```

Output:

```
Daily claims: [80, 120, 130, 95, 101, 150, 160, 170, 100, 105, 110, 99]
Longest run: 4 days
From day 5 to day 8
```

Explanation: The window [left, right] always contains only days with more than 100 claims. The right pointer moves one day at a time. If the day has 100 claims or fewer, it does not exceed 100, so the window is emptied by moving left to right + 1. Otherwise the window grows, and its length is compared with the best length found so far. Days 5 to 8 (101, 150, 160, 170) form the longest run; day 9 has exactly 100, which does not exceed 100, so the run stops there.

Complexity: O(n) time, because each day is visited once; O(1) extra space.

Task 5: Merging Two Sorted Claim Lists Using Two Pointers

Sample Input:

Office A: 1002, 1015, 1020, 1040, 1055

Office B: 1008, 1015, 1033, 1040, 1070

Program:

```
def merge_sorted(a, b):
    i = 0
    j = 0
    result = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i])
            i += 1
        else:
            result.append(b[j])
            j += 1
    while i < len(a):
        result.append(a[i])
        i += 1
    while j < len(b):
        result.append(b[j])
        j += 1
    return result

office_a = [1002, 1015, 1020, 1040, 1055]
office_b = [1008, 1015, 1033, 1040, 1070]
print("Office A:", office_a)
print("Office B:", office_b)
```

```
print("Merged:", merge_sorted(office_a, office_b))
```

Output:

```
Office A: [1002, 1015, 1020, 1040, 1055]
Office B: [1008, 1015, 1033, 1040, 1070]
Merged: [1002, 1008, 1015, 1015, 1020, 1033, 1040, 1040, 1055, 1070]
```

Explanation: One pointer (i) walks through Office A and another (j) walks through Office B. At each step the smaller of the two current claim IDs is copied to the result and its pointer moves forward. When one list is finished, the rest of the other list is copied. Because both lists are already sorted, no extra sorting is needed. Equal IDs (1015 and 1040) are both kept, with Office A first.

Complexity: O(m + n) time and O(m + n) space for the merged list, where m and n are the lengths of the two lists.

Deliverables

Test Cases

The test program below is placed after the functions of Tasks 1 to 5 in the same file. It covers empty input and duplicates for every task.

```
def check(name, actual, expected):
    status = "PASS" if actual == expected else "FAIL"
    print(status, "-", name)

# Task 1 - BST
root = None
check("T1 empty tree inorder", inorder(root), [])
check("T1 search in empty tree", search(root, 5), False)
for k in [10, 5, 10, 15, 5]:
    root = insert(root, k)
check("T1 duplicate IDs ignored", inorder(root), [5, 10, 15])

# Task 2 - Hashing
dups, _ = find_duplicates([])
check("T2 empty input", dups, [])
dups, _ = find_duplicates([(1, "AB 1", "2026-01-01"), (2, "ab1", "2026-01-01")])
check("T2 case and spacing duplicate", dups, [(2, 1)])
dups, _ = find_duplicates([(1, "AB1", "2026-01-01"), (2, "AB1", "2026-01-02")])
check("T2 different date is not duplicate", dups, [])

# Task 3 - Graph
check("T3 empty graph path", shortest_path({}, "A", "B"), None)
check("T3 same start and goal", shortest_path(graph, "Assessment", "Assessment"), ["Assessment"])
check("T3 empty graph loop", find_loop({}), None)
check("T3 self loop", find_loop({"A": ["A"]}), ["A", "A"])

# Task 4 - Sliding window
check("T4 empty list", longest_run([]), (0, -1, -1))
check("T4 no day above 100", longest_run([100, 90, 100]), (0, -1, -1))
check("T4 all days above 100", longest_run([101, 150, 200]), (3, 1, 3))
```

```
check("T4 equal runs keep first", longest_run([110, 50, 120]), (1, 1, 1))

# Task 5 - Two pointers
check("T5 both empty", merge_sorted([], []), [])
check("T5 one list empty", merge_sorted([], [3, 4]), [3, 4])
check("T5 duplicates kept", merge_sorted([1, 2, 2], [2, 3]), [1, 2, 2, 2, 3])
```

Output:

```
PASS - T1 empty tree inorder
PASS - T1 search in empty tree
PASS - T1 duplicate IDs ignored
PASS - T2 empty input
PASS - T2 case and spacing duplicate
PASS - T2 different date is not duplicate
PASS - T3 empty graph path
PASS - T3 same start and goal
PASS - T3 empty graph loop
PASS - T3 self loop
PASS - T4 empty list
PASS - T4 no day above 100
PASS - T4 all days above 100
PASS - T4 equal runs keep first
PASS - T5 both empty
PASS - T5 one list empty
PASS - T5 duplicates kept
```

Complexity Table

Task	Solution	Time	Space
1	BST insert / search	O(log n) average, O(n) worst	O(n)
1	BST traversals	O(n)	O(h) stack
2	Hash table with chaining	O(1) average per claim	O(n)
3	BFS shortest path	O(V + E)	O(V)
3	DFS loop detection	O(V + E)	O(V)
4	Sliding window	O(n)	O(1)
5	Two-pointer merge	O(m + n)	O(m + n)

Day 1 – Problem Solving Techniques and Data Structures – I

Set 3 – Logistics Network and Delivery Windows

Business Scenario	A courier company is optimising hub-to-hub routing and delivery-window planning across a national network.
Tasks	1. Store hub codes in a BST and implement search and insert; identify the height of the tree for a given insertion order and explain the effect of sorted input. 2. Build a lookup of parcel tracking numbers using a hash map; implement a check that reports parcels scanned twice at the same hub. 3. Represent 10 hubs and their direct routes as an adjacency list; find the route with the fewest hops between two hubs (BFS) and list every hub reachable from the central depot (DFS). 4. Given hourly parcel counts, find the 4-hour window with the highest average using a sliding window. 5. Use two pointers to check whether a delivery route description is a palindrome of hub codes, ignoring case and separators.
Deliverables	Code, graph diagram or adjacency list used, output and a short explanation of every algorithm choice.

Task 1: Binary Search Tree (BST) of Hub Codes

Sample Input:

Hub codes (insertion order): HYD, DEL, MUM, BLR, CHN, KOL, PUN, NGP, JPR, AMD

Height is counted as the number of levels on the longest path from the root to a leaf (an empty tree has height 0).

Program:

```
class Node:
    def __init__(self, code):
        self.code = code
        self.left = None
        self.right = None

def insert(root, code):
    if root is None:
        return Node(code)
    if code < root.code:
        root.left = insert(root.left, code)
    elif code > root.code:
        root.right = insert(root.right, code)
    return root

def search(root, code):
    while root is not None:
        if code == root.code:
            return True
        elif code < root.code:
            root = root.left
```

```
        else:
            root = root.right
    return False

def height(root):
    if root is None:
        return 0
    return 1 + max(height(root.left), height(root.right))

def build(codes):
    root = None
    for code in codes:
        root = insert(root, code)
    return root

hubs = ["HYD", "DEL", "MUM", "BLR", "CHN", "KOL", "PUN", "NGP", "JPR", "AMD"]

root = build(hubs)
print("Insertion order:", hubs)
print("Height (levels):", height(root))
print("Search PUN:", search(root, "PUN"))
print("Search GOA:", search(root, "GOA"))

sorted_hubs = sorted(hubs)
print("Sorted order:", sorted_hubs)
print("Height (levels):", height(build(sorted_hubs)))
```

Output:

```
Insertion order: ['HYD', 'DEL', 'MUM', 'BLR', 'CHN', 'KOL', 'PUN', 'NGP', 'JPR', 'AMD']
Height (levels): 4
Search PUN: True
Search GOA: False
Sorted order: ['AMD', 'BLR', 'CHN', 'DEL', 'HYD', 'JPR', 'KOL', 'MUM', 'NGP', 'PUN']
Height (levels): 10
```

Explanation: Hub codes are compared alphabetically: a smaller code goes to the left and a larger code to the right. For the given insertion order the root is HYD and the longest paths (for example HYD → DEL → BLR → CHN) have 4 nodes, so the height is 4. When the codes are inserted in sorted order, every new code is larger than all codes already stored, so it always goes to the right. The tree becomes a single chain like a linked list with height 10 (equal to the number of hubs), and search degrades from O(log n) to O(n). Self-balancing trees (AVL or Red-Black) or a shuffled insertion order avoid this problem.

Complexity: insert and search take O(h) time, which is O(log n) for a balanced tree and O(n) for a skewed tree; computing the height takes O(n); space is O(n) for the nodes plus O(h) for the recursion stack.

Task 2: Parcel Tracking Lookup Using Hash Map

Sample Input:

Scan log (TrackingNo, Hub, ScanTime):

TrackingNo	Hub	ScanTime
PK1001	DEL	2026-11-01 08:10
PK1002	MUM	2026-11-01 08:15
PK1001	MUM	2026-11-01 14:30
PK1003	BLR	2026-11-01 09:00
PK1001	DEL	2026-11-01 15:45
PK1002	MUM	2026-11-01 08:20
PK1003	HYD	2026-11-01 18:05
PK1004	CHN	2026-11-01 10:10

Program:

```
scans = [
    ("PK1001", "DEL", "2026-11-01 08:10"),
    ("PK1002", "MUM", "2026-11-01 08:15"),
    ("PK1001", "MUM", "2026-11-01 14:30"),
    ("PK1003", "BLR", "2026-11-01 09:00"),
    ("PK1001", "DEL", "2026-11-01 15:45"),
    ("PK1002", "MUM", "2026-11-01 08:20"),
    ("PK1003", "HYD", "2026-11-01 18:05"),
    ("PK1004", "CHN", "2026-11-01 10:10"),
]

def build_lookup(scans):
    tracking = {}
    for parcel, hub, time in scans:
        if parcel not in tracking:
            tracking[parcel] = []
        tracking[parcel].append((hub, time))
    return tracking

def find_double_scans(tracking):
    report = []
    for parcel, history in tracking.items():
        hub_count = {}
        for hub, time in history:
            hub_count[hub] = hub_count.get(hub, 0) + 1
        for hub, count in hub_count.items():
            if count >= 2:
                report.append((parcel, hub, count))
    return report

tracking = build_lookup(scans)

print("Lookup PK1001:", tracking.get("PK1001"))
print("Lookup PK9999:", tracking.get("PK9999", "Not found"))
```

```
for parcel, hub, count in find_double_scans(tracking):
    print("Parcel", parcel, "scanned", count, "times at hub", hub)
```

Output:

```
Lookup PK1001: [('DEL', '2026-11-01 08:10'), ('MUM', '2026-11-01 14:30'), ('DEL', '2026-11-01 15:45')]
Lookup PK9999: Not found
Parcel PK1001 scanned 2 times at hub DEL
Parcel PK1002 scanned 2 times at hub MUM
```

Explanation: A Python dictionary (hash map) maps each tracking number to the list of its scans, so a parcel can be looked up directly without searching the whole log. To find double scans, every parcel's history is read once and a second dictionary counts how many times each hub appears; any hub with a count of 2 or more is reported. PK1001 was scanned twice at DEL and PK1002 twice at MUM. PK1001 also visited MUM once, which is correctly not reported.

Complexity: building the lookup takes O(n) time for n scans; a lookup takes O(1) on average; the double-scan check takes O(n) in total; space is O(n).

Task 3: Hub Network Using BFS and DFS

Sample Input:

Central depot: NGP. The 10 hubs (DEL – Delhi, MUM – Mumbai, BLR – Bengaluru, CHN – Chennai, HYD – Hyderabad, KOL – Kolkata, PUN – Pune, NGP – Nagpur, AMD – Ahmedabad, JPR – Jaipur) and their direct two-way routes are shown below as an adjacency list.

Hub	Direct routes to
NGP (central depot)	DEL, MUM, HYD, KOL
DEL	NGP, JPR
MUM	NGP, AMD, PUN
BLR	HYD, CHN
CHN	BLR, KOL
HYD	NGP, PUN, BLR
KOL	NGP, CHN
PUN	MUM, HYD
AMD	JPR, MUM
JPR	DEL, AMD

Program:

```
from collections import deque

routes = [
    ("NGP", "DEL"), ("NGP", "MUM"), ("NGP", "HYD"), ("NGP", "KOL"),
    ("DEL", "JPR"), ("JPR", "AMD"), ("AMD", "MUM"), ("MUM", "PUN"),
    ("PUN", "HYD"), ("HYD", "BLR"), ("BLR", "CHN"), ("CHN", "KOL"),
]

hubs = ["NGP", "DEL", "MUM", "BLR", "CHN", "HYD", "KOL", "PUN", "AMD", "JPR"]

adj = {hub: [] for hub in hubs}
```

```
for a, b in routes:
    adj[a].append(b)
    adj[b].append(a)

def fewest_hops(adj, start, goal):
    parent = {start: None}
    queue = deque([start])
    while queue:
        hub = queue.popleft()
        if hub == goal:
            path = []
            while hub is not None:
                path.append(hub)
                hub = parent[hub]
            return path[::-1]
        for nxt in adj[hub]:
            if nxt not in parent:
                parent[nxt] = hub
                queue.append(nxt)
    return None

def reachable(adj, depot):
    visited = []
    def dfs(hub):
        visited.append(hub)
        for nxt in adj[hub]:
            if nxt not in visited:
                dfs(nxt)
    dfs(depot)
    return visited

print("Adjacency list:")
for hub in hubs:
    print(" ", hub, "->", adj[hub])

path = fewest_hops(adj, "DEL", "BLR")
print("Fewest hops DEL -> BLR:", path, "-", len(path) - 1, "hops")

path = fewest_hops(adj, "AMD", "CHN")
print("Fewest hops AMD -> CHN:", path, "-", len(path) - 1, "hops")

found = reachable(adj, "NGP")
print("Reachable from NGP (DFS):", found)
print("Total reachable:", len(found))
```

Output:

```
Adjacency list:
  NGP -> ['DEL', 'MUM', 'HYD', 'KOL']
  DEL -> ['NGP', 'JPR']
  MUM -> ['NGP', 'AMD', 'PUN']
  BLR -> ['HYD', 'CHN']
  CHN -> ['BLR', 'KOL']
  HYD -> ['NGP', 'PUN', 'BLR']
  KOL -> ['NGP', 'CHN']
  PUN -> ['MUM', 'HYD']
  AMD -> ['JPR', 'MUM']
  JPR -> ['DEL', 'AMD']
Fewest hops DEL -> BLR: ['DEL', 'NGP', 'HYD', 'BLR'] - 3 hops
Fewest hops AMD -> CHN: ['AMD', 'MUM', 'NGP', 'KOL', 'CHN'] - 4 hops
Reachable from NGP (DFS): ['NGP', 'DEL', 'JPR', 'AMD', 'MUM', 'PUN', 'HYD', 'BLR', 'CHN', 'KOL']
Total reachable: 10
```

Explanation: An adjacency list is chosen because the network is sparse (12 routes among 10 hubs); it stores only the routes that exist, instead of a 10 × 10 table. Each route is two-way, so it is added to both hubs. BFS is chosen for the fewest hops because it explores hubs in order of increasing distance (all hubs 1 hop away, then 2 hops, and so on), so the first time it reaches the destination is the shortest route. DEL → BLR needs 3 hops (DEL, NGP, HYD, BLR). DFS is chosen to list every reachable hub because it follows each route to its end and marks hubs as visited; starting at NGP it reaches all 10 hubs. A hub with no route chain to NGP would not appear in this list.

Complexity: BFS and DFS both take O(V + E) time and O(V) space, where V is the number of hubs and E the number of routes.

Task 4: Highest-Average 4-Hour Window Using Sliding Window

Sample Input:

Hourly parcel counts from 08:00 to 19:00: 120, 135, 150, 160, 155, 140, 180, 190, 185, 170, 130, 125

Program:

```
def best_window(counts, k=4):
    if len(counts) < k:
        return None
    window_sum = sum(counts[:k])
    best_sum = window_sum
    best_start = 0
    for i in range(k, len(counts)):
        window_sum = window_sum + counts[i] - counts[i - k]
        if window_sum > best_sum:
            best_sum = window_sum
            best_start = i - k + 1
    return best_start, best_sum / k

hourly = [120, 135, 150, 160, 155, 140, 180, 190, 185, 170, 130, 125]
first_hour = 8
```

```
start, avg = best_window(hourly, 4)
window = hourly[start:start + 4]
print("Hourly counts:", hourly)
print("Best window:", window)
print("Hours:", str(first_hour + start) + ":00 to " + str(first_hour + start + 4) + ":00")
print("Average:", avg)
```

Output:

```
Hourly counts: [120, 135, 150, 160, 155, 140, 180, 190, 185, 170, 130, 125]
Best window: [180, 190, 185, 170]
Hours: 14:00 to 18:00
Average: 181.25
```

Explanation: The sum of the first 4 hours is calculated once. The window is then moved one hour at a time: the new hour is added and the hour that leaves the window is subtracted, so nothing is added again from the start. Because every window has 4 hours, the window with the highest sum also has the highest average. The best window is 14:00 to 18:00 (180, 190, 185, 170) with an average of 181.25 parcels per hour.

Complexity: O(n) time instead of O(n × k) for re-adding every window; O(1) extra space.

Task 5: Route Palindrome Check Using Two Pointers

Sample Input:

Route descriptions: "DEL-MUM-BLR-MUM-DEL", "del > mum > blr > MUM > Del", "DEL-MUM-BLR-CHN", "MUM-DEL-LED-MUM", "HYD" and an empty string.

Program:

```
import re

def is_palindrome_hubs(route):
    hubs = re.findall("[A-Za-z0-9]+", route.lower())
    left = 0
    right = len(hubs) - 1
    while left < right:
        if hubs[left] != hubs[right]:
            return False
        left += 1
        right -= 1
    return True

def is_palindrome_chars(route):
    left = 0
    right = len(route) - 1
    while left < right:
        if not route[left].isalnum():
            left += 1
        elif not route[right].isalnum():
            right -= 1
        elif route[left].lower() != route[right].lower():
            return False
        else:
```

```
            left += 1
            right -= 1
    return True

routes = [
    "DEL-MUM-BLR-MUM-DEL",
    "del > mum > blr > MUM > Del",
    "DEL-MUM-BLR-CHN",
    "MUM-DEL-LED-MUM",
    "HYD",
    "",
]
for r in routes:
    print(repr(r))
    print("   hub-level:", is_palindrome_hubs(r), "| letter-level:", is_palindrome_chars(r))
```

Output:

```
  'DEL-MUM-BLR-MUM-DEL'
    hub-level: True | letter-level: False
  'del > mum > blr > MUM > Del'
    hub-level: True | letter-level: False
  'DEL-MUM-BLR-CHN'
    hub-level: False | letter-level: False
  'MUM-DEL-LED-MUM'
    hub-level: False | letter-level: True
  'HYD'
    hub-level: True | letter-level: False
  ''
    hub-level: True | letter-level: True
```

Explanation: is_palindrome_hubs() answers the business question: does the route read the same forwards and backwards as a sequence of hub codes? Separators (-, >, spaces) are dropped and the codes are converted to lower case, then one pointer starts at the first hub and another at the last hub; they are compared and moved towards each other until they meet. A route such as DEL-MUM-BLR-MUM-DEL is a palindrome of hub codes. is_palindrome_chars() is the letter-by-letter version: its two pointers skip any character that is not a letter or digit and compare the remaining letters ignoring case, so "MUM-DEL-LED-MUM" is a palindrome when read letter by letter. An empty route is treated as a palindrome.

Complexity: O(n) time, because each character or hub is examined at most once; O(n) space for the hub list in the hub-level version and O(1) for the letter-level version.

Deliverables

Graph Representation

The adjacency list used in Task 3 is given in the sample input of Task 3 and printed by the program output above.

Algorithm Choices

Task	Technique	Reason for choice	Time	Space
1	BST	Ordered storage with fast search and insert; sorted input gives height n	O(h)	O(n)
2	Hash map	Direct lookup by tracking number; counting scans per hub	O(1) avg lookup, O(n) check	O(n)
3	Adjacency list, BFS, DFS	Sparse network; BFS gives fewest hops; DFS reaches every hub	O(V + E)	O(V)
4	Sliding window	Reuses the previous sum instead of re-adding 4 values	O(n)	O(1)
5	Two pointers	Compares both ends moving inwards with no reversed copy	O(n)	O(n) / O(1)